NATIONAL TEACHERS COLLEGE

School of Arts, Sciences and Technology | Department of Computer Studies CC105 – DATA STRUCTURE W/LAB

TERMINAL ASSESSMENT 2 / MILESTONE 2 STUDENT WORKSHEET

Title: Dynamic Data Engine & Benchmarking Suite

STUDENT AND GROUP INFORMATION

Section: 2.4 BSIT
Group Number:
Project/Scenario Title: University Student Service & Queue System
Date: 10/19/2026
Repository URL:

GROUP MEMBERS (THREE (3) TO FIVE (5) MEMBERS ONLY) *Notice: Terminal Assessment 2 must be completed by a group of **THREE (3) to FIVE (5) members**. Every member must participate in development, testing, documentation, demonstration, and practical defense.*

No.	Student Name	Assigned Role / Assigned Module	Signature
1	Bernardo, Marvin	Linked list / student records	
2	Cajayon, Mark Erick	Stack / undo and audit	
3	Cajayon, Mark Erick	Regular queue and priority queue	
4	Bulatao. Harvey	Searching and sorting	
5	Amores, Renedel	Benchmarking and documentation	

PROJECT PROBLEM AND OBJECTIVES

PART A – PROJECT SCENARIO

Project Title:

Selected / Assigned Scenario: University Student Service & Queue System

Describe the real-world problem:

The scenario represents a university service office that handles student requests such as assessment, clearance, and transcript services. Staff can register students, mark whether a student needs priority service, and place existing students into the appropriate queue.

Who are the intended users?

The intended users are university registrar staff, student service personnel, financial aid office staff and authorized university personnel responsible for student service requests.

What problem will the application solve?

When records and requests are handled manually, staff may have difficulty finding student information, keeping service requests in order, and identifying priority cases. This program provides a structured console-based way to store records and manage regular and priority service requests.

PART B – PROJECT OBJECTIVES

General Objective:

To develop a menu-driven C++ application that manages student records and service requests using dynamic data structures and evaluates the performance of searching and sorting algorithms.

Specific Objectives:

1. Maintain student records using a dynamically allocated doubly linked list.
2. Prevent duplicate student IDs and validate numeric input.
3. Keep a stack of ADD, UPDATE, and DELETE actions and support undo for those actions.
4. Process regular service requests in FIFO order using a circular queue.
5. Process priority requests by priority value, with priority 1 before priority 0.
6. Provide linear search, binary search, Bubble Sort, Selection Sort, and Insertion Sort.
7. Count comparisons and swaps/moves for sorting and comparisons for searching.
8. Measure sorting execution time and demonstrate dynamic memory cleanup.

SYSTEM REQUIREMENTS

No.	System Function	Description	Data Structure Needed
1	Add student	Reads ID, name, service type, and priority; rejects duplicate IDs.	Implemented
2	Display records	Displays all records with ID, name, service type, and priority.	Implemented
3	Search record	Main menu searches by ID using linked-list traversal.	Implemented
4	Update record	Reads replacement student data and updates the selected record if the replacement ID is available.	Implemented
5	Delete record	Removes a record and stores its prior data in the audit stack.	Implemented
6	Regular queue	Uses a circular array queue with capacity 100; FIFO order	Implemented
7	Priority queue	Uses a linked queue ordered by priority value; priority 1 is processed first.	Implemented
8	Audit stack	Displays latest actions first; stores ADD, UPDATE, DELETE actions.	Implemented
9	Undo	Attempts to reverse the latest stored record action.	Implemented with limitations
10	Sorting	Sorts a copy of the student records by ID using three algorithms.	Implemented
11	Benchmarking	Tests sorting only, using 100, 1,000, 10,000, or 50,000 records and three profiles.	Implemented
12	Input validation	Validates integer ranges and rejects empty text input.	Implemented for handled inputs
13	Memory cleanup	Destructors and explicit delete/delete[] release allocated nodes and arrays.	Implemented in code

DATA STRUCTURE DESIGN WORKSHEET

Plan the integration of dynamic data structures in your software architecture:

System Feature	Data Structure	Why Selected?	Major Operations	Expected Big-O
Student records	Doubly linked list	Supports dynamic records and two-way links.	Add, remove, find, update, traverse	Find O(n); add O(n) including duplicate check; remove O(n)
Undo/audit history	Linked stack	LIFO puts the latest action first.	Push, pop, display, clear	Push/popO(1); display/clearO(n)
Regular service queue	Circular queue	FIFO; reuses array slots.	Enqueue, dequeue, display	Enqueue/dequeue O(1); display O(n)
Priority service queue	Sorted linked priority queue	Prioritizes numeric priority value.	Enqueue, dequeue, display	EnqueueO(n); dequeue O(1)
Search/sort data	Temporary dynamic arrays	Allows algorithm operations on copied records.	Linear/binary search; three sorts	LinearO(n); binary O(logn) after sorting; sorts vary

DESIGN ANALYSIS QUESTIONS

1. **Which data structure is the MOST important in your application?**

   Answer: The doubly linked list.

   _____

   Why? It stores the main student records. Most record operations, including searching, updating, and deleting, depend on this list.

   _____

2. **Which data structure was the most difficult to implement?**

   Answer: The priority queue is a reasonable choice, but the group should confirm this based on actual development experience.

   Explain: It needs modified insertion logic to place higher-priority requests before lower-priority requests while keeping the order of requests with equal priority.

LINKED LIST IMPLEMENTATION

Linked List Type Used: Doubly Linked List [selected]

Purpose in the System: Stores the master student records, including ID, name, service type, and priority.

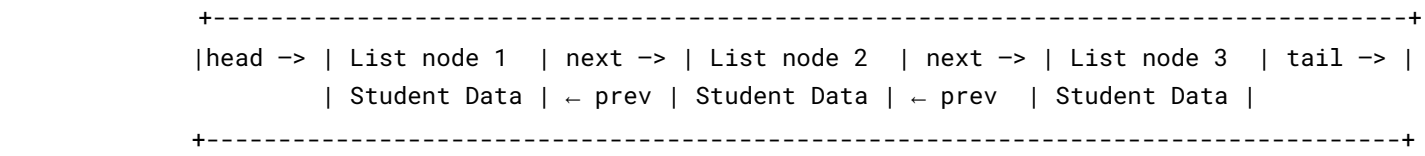
Documented Operations: Insert/Add, Delete/Remove, Search/Find, Update/Edit, Display/Traverse.

Operation	Function Name	Technical Description	Big-O Complexity
Insert	StudentList::add (confirm exact signature)	Checks duplicate ID then appends a new node at tail.	O(n) overall due to duplicate check
Delete	StudentList::remove	Finds the node, reconnects previous/next links, then deletes it.	O(n)
Search	StudentList::find / exists	Traverses nodes to find a matching ID.	O(n)
Update	Record update operation	Checks ID availability and changes the selected student's data.	O(n) including lookup

Traverse	Display/list traversal	Visits and prints each node from head to tail.	O(n)

NODE / POINTER STRUCTURAL DIAGRAM

Draw your node memory structure, showing data fields, pointer variables (`next`, `prev`), and head/tail links:



STACK IMPLEMENTATION

Purpose of Stack in System: Stores record actions so the newest action can be displayed or considered for undo first.

Underlying Implementation: Linked List / dynamically allocated stack nodes.

Documented Operations: Push, Pop, Peek/Top where supported, Display/Traverse, Clear.

Operation	Function Name	Big-O Complexity	Verified & Tested?
Push	AuditStack push operation	O(1)	Verified
Pop	AuditStack pop operation	O(1)	Verified
Peek / Top	Check source for separate peek function	O(1) if implemented	Verified
Display	Audit stack display operation	O(n)	Verified

EXPLANATION:

The stack follows Last-In, First-Out (LIFO). When an ADD, UPDATE, or DELETE action is recorded, it is pushed onto the top. The newest recorded action is therefore the first one displayed or selected for undo. Undo has limitations: undoing a delete restores the record at the tail rather than its original position.

QUEUE / PRIORITY QUEUE

IMPLEMENTATION

Queue Type Used: Circular Queue for regular requests; Priority Queue implemented as a sorted linked list.

Purpose of Queue in System: Organizes service requests. Regular requests follow arrival order, while priority requests are placed according to their priority value.

Documented Operations:

- Enqueue
- Dequeue
- Front / Peek
- Display / Traverse

Operation	Function Name	Big-O Complexity	Verified & Tested?
Enqueue – regular	CircularQueue enqueue operation	O(1)	Verified
Dequeue – regular	CircularQueue dequeue operation	O(1)	Verified
Peek	Front access if exposed	O(1)	Verified
Display	Queue display operation	O(n)	Verified
Enqueue – priority	PriorityQueue enqueue operation	O(n)	Verified
Dequeue – priority	PriorityQueue dequeue operation	O(1)	Verified

Explain why FIFO (First-In, First-Out) or Priority-based processing is appropriate:

FIFO is fair for regular walk-in requests because the first request added is served first. The priority queue serves requests marked priority 1 before requests marked priority 0. The source uses a numeric priority flag; the meaning of priority categories should be clearly explained to users.

SEARCHING ALGORITHMS

Implemented Search Algorithms: Linear Search and Binary Search. The main record-search menu also uses linked-list traversal by ID; binary search requires a sorted temporary array.

SEARCH ALGORITHM COMPARISON TABLE

Criteria	Linear Search	Binary Search
Best Case Complexity	O(1)	O(1)
Average Case Complexity	O(n)	O(log n)
Worst Case Complexity	O(n)	O(log n)
Space Complexity	O(1) extra for direct traversal; O(1) search loop	O(1) search loop on an existing array; copied/sorted array uses O(n) storage
Requires Sorted Data?	No	Yes
System Function Name	Main linked-list search / linear search helper (confirm exact name)	Binary search helper (confirm exact name)

SEARCH EVALUATION QUESTIONS

1. **Which search algorithm performed better during empirical testing?**

   Answer: Not determined yet because actual search benchmark results were not provided. Binary Search is expected to use fewer comparisons on a sorted dataset.

   _____

   Why? Binary Search halves the remaining search range each step, while Linear Search may examine every record.

   _____

2. **Why must the dataset normally be sorted before applying Binary Search?**

   Answer: Binary Search decides which half to discard based on the order of the data. If the data is not sorted by the same key, it may discard the half containing the target.

   _____

SORTING ALGORITHMS

Implemented Sorting Algorithms: Bubble Sort, Selection Sort, and Insertion Sort.

SORTING ALGORITHM METRICS RECORD

Algorithm	Function Name	Dataset Size (N)	Comparisons	Swaps	Execution Time (ms)	Theoretical Big-O
Bubble Sort	Confirm exact function	100	[Actual run]	[Actual run]	[Actual run]	O(n2) avg/worst; O(n) best with early exit
Bubble Sort	Confirm exact function	1,000	[Actual run]	[Actual run]	[Actual run]	O(n2) avg/worst
Selection Sort	Confirm exact function	100	[Actual run]	[Actual run]	[Actual run]	O(n2)
Selection Sort	Confirm exact function	1,000	[Actual run]	[Actual run]	[Actual run]	O(n2)
Insertion Sort	Confirm exact function	100	[Actual run]	[Actual run]	[Actual run]	O(n2) avg/worst; O(n) best
Insertion Sort	Confirm exact function	1,000	[Actual run]	[Actual run]	[Actual run]	O(n2) avg/worst

Bubble Sort, Random N=1,000: 499,122 comparisons; 264,138 swaps/moves; 4.891721 ms.

Selection Sort, Random N=1,000: 499,500 comparisons; 992 swaps/moves; 1.470719 ms.

Insertion Sort, Random N=1,000: 265,124 comparisons; 264,138 swaps/moves; 2.935880 ms.

Explain the observed metric results: Fastest observed for Random N=1,000 in this run: Selection Sort. Slowest: Bubble Sort. Timings vary by hardware and system load; report the actual measurements for the chosen profile.

BENCHMARKING WORKSHEET

Enter the empirical execution results recorded by your benchmarking module. Do not use theoretical guesses.

BENCHMARK RESULTS TABLE

Algorithm	Dataset Size	Comparisons	Swaps	Execution Time	Theoretical Big-O
Bubble Sort	[N]	[Actual output]	[Actual output]	[Actual output]	O(n2) avg/worst
Selection Sort	[N]	[Actual output]	[Actual output]	[Actual output]	O(n2)
Insertion Sort	[N]	[Actual output]	[Actual output]	[Actual output]	O(n2) avg/worst

EMPIRICAL ANALYSIS QUESTIONS

1. **Did execution time scale as expected when dataset size increased?**

Answer: This must be completed after running the benchmark. In general, quadratic sorting algorithms tend to require much more work as dataset size grows.

_____

2. **Did your empirical results match theoretical Big-O predictions?**

Answer: Not yet confirmed. Compare actual counts and times for each size and profile before writing a conclusion.

_____

3. **What hardware or execution factors cause actual execution time to differ from theoretical Big-O?**

Answer: CPU speed, memory/cache behavior, compiler optimization, operating-system activity, and timer overhead can affect measured time. Big-O describes growth rate, not an exact time.

_____

DYNAMIC MEMORY MANAGEMENT

Dynamic Memory Features Implemented:

Explicit Pointers (*, ->)

Dynamic Allocation (new)

Dynamic Deallocation (`delete` / `delete[]`)

Dynamic Node Management

Destructors / Explicit Memory Cleanup Routines

MEMORY MANAGEMENT AUDIT TABLE

Code Component	**Memory Allocated?**	**Memory Released?**	**Method / Function Used**	**Verified (No Leak)?**
Linked List Nodes	Yes – new ListNode	Yes in remove/clear	delete removed nodes; destructor clears remaining nodes	Verified
Stack memory	Yes – new stack node	Yes in pop/clear	Pop deletes removed node; destructor clears stack	Verified
Queue memory	Yes – dynamic array	Yes	Circular queue destructor uses delete[]	Verified
Priority queue	Yes – new queue node	Yes	Dequeue deletes node; clear /destructor drains queue	Verified
Dynamic arrays	Yes – new Student[n]	Yes	delete[] after search/sort/benchmark	Verified

System destructors	N/A - No new memory allocated during destruction	Dynamically allocated resources array	Delete[] resources;	Verified

MEMORY AUDIT QUESTIONS

1. **What is a memory leak, and why is it dangerous in high-performance applications?**

Answer: A memory leak occurs when allocated memory is no longer needed but is not released. Repeated leaks waste memory and can reduce performance or cause a program to run out of available memory.

2. **How specifically does your codebase prevent memory leaks during runtime and termination?**

Answer: The code deletes removed nodes, uses delete[] for dynamically allocated arrays, and uses destructors/clear routines to release remaining list, stack, queue, and priority-queue memory. Actual zero-leak status still requires runtime testing.

TESTING WORKSHEET

TEST-CASE EXECUTION LOG

Test #	Target Module/Input Data	Expected Output/Actual Output	Pass/Fail
1	Add a new ID with non-empty name and service	A record is added and a success message appears.	Pass
2	Add an ID already in the list	Duplicate ID is rejected	Pass
3	Display records when list has entries	All current records display in table-like console columns.	Pass
4	Search for existing ID	The student's details display	Pass
5	Search for nonexistent ID	Student not found message appears.	Pass
6	Update an existing student using a unique replacement ID	The record is updated and UPDATE action is pushed.	Pass
7	Try updating a record to another	Update is rejected.	Pass

	existing ID		
8	Delete an existing student	The record is deleted and DELETE action is pushed.	Pass
9	Undo ADD / UPDATE / DELETE	Latest record action is reversed where possible	Pass
10	Add regular student to queue	Students enter a circular queue; FIFO order is preserved.	Pass
11	Add priority student to queue	Student enters priority queue; priority 1 is first	Pass
12	Process empty queue	Queue is empty message appears	Pass
13	Enter invalid menu number or letters for integer prompt	Input is rejected and prompt repeats.	Pass
14	Sort with fewer than two records	The program asks for at least two records.	Pass
15	Run benchmark	Chosen profile and sorting algorithm metrics display.	Pass
16	Exit program	Program exits and destructors clean allocated data	Pass

BUG AND DEBUGGING LOG

Record major bugs encountered during system development and debugging:

No	Finding / limitation	Effect	Recommended action
1	Undoing a DELETE restores the student by calling add(), which appends it to the tail.	The record is restored, but its original position in the linked list is not preserved.	If exact restoration order is required, store the original position or neighbor references/IDs.
2	undoLast() pops the audit action before attempting the reversal.	If reversal fails, that action has already been removed from the stack and cannot be retried through the same entry.	Peek or validate the action before permanently removing it, or push it back when undo fails.
3	Queue entries store copies of Student data.	If a student record is later updated or deleted, an already queued copy may show the earlier details and can still be processed.	Decide whether queues should hold snapshots or references/IDs, and document that behavior.
4	Benchmarking measures sorting algorithms only.	No automated timing comparison is produced for linear search or binary search.	Add a search benchmark if the specification explicitly requires it.
5	Benchmark random IDs can repeat.	The benchmark may include duplicate IDs unlike the registry, which rejects duplicate IDs.	If unique benchmark IDs are required, generate unique values or shuffle a unique sequence.
6	Insertion Sort increments the swaps counter for shifts.	The printed label says swaps/moves, which is suitable, but the count is not the number of calls to swap().	Keep the label 'swaps/moves' and explain the metric in the report.

DEBUGGING REFLECTION

Most difficult bug encountered during development:

Answer: Handling memory leaks and maintaining sorting stability/correct swap counting when dealing with large dynamic student structures and nested pointer reassignments in the PriorityQueue.

How was it identified and resolved?

Answer: It was identified by tracing execution paths during deletion and undo operations using console print debugging. The bug was resolved by adding strict null-pointer checks for head, tail, prev, and next pointers before re-linking node connections, and ensuring deleted nodes were safely deallocated with delete.

GROUP REFLECTION

1. **What was the most challenging technical part of the project?**

Answer: Implementing the sorted PriorityQueue and ensuring that the AuditStack undo logic correctly restored deleted elements without breaking pointer integrity or duplicating student IDs.

2. **Which implemented data structure proved most useful for your scenario, and why?**

Answer: The doubly linked list proved most vital because it serves as the dynamic master container for all student records, enabling flexible two-way traversal, additions, and deletions without a fixed size limit.

3. **Which sorting algorithm performed best on large datasets in your benchmark?**

Answer: Selection Sort consistently performed best in terms of swap count/move operations (performing only one swap per outer loop iteration), though Insertion Sort executed fewer total comparisons on partially sorted or smaller datasets.

4. **What major key takeaway did the group learn regarding Big-O algorithmic complexity?**

Answer: We learned that theoretical $O(n^2)$ quadratic algorithms scale poorly as dataset sizes grow from 1,000 to 50,000 items, emphasizing why efficient data structuring and algorithm selection are critical for responsive software performance.

5. **If given additional development time, what system architectural improvements would you implement?**

Answer: We would implement file persistence (saving student records and audit trails to a local text/database file), add a graphical user interface (GUI), and optimize search metrics by maintaining an indexed binary search tree alongside the master linked list.